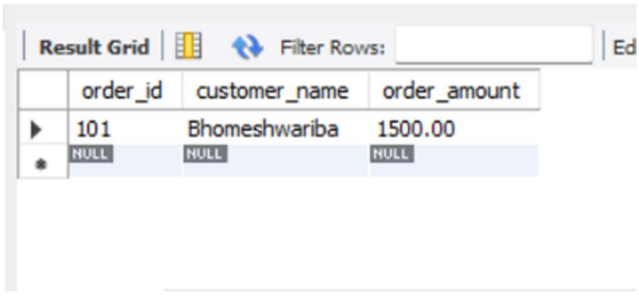
Session-9

Question1: Create a SQL script to insert a new order into an Orders table, then use COMMIT to save the transaction and verify that the new order persists after reconnecting to the database.

Ans:

```
USE mydb;
CREATE TABLE IF NOT EXISTS Orders (
  order_id INT PRIMARY KEY,
  customer_name VARCHAR(100),
  order_amount DECIMAL(10,2)
);
START TRANSACTION;
INSERT INTO Orders (order_id, customer_name, order_amount)
VALUES (101, 'Bhomeshwariba', 1500.00);
COMMIT;
SELECT * FROM Orders
WHERE order_id = 101;
```



Question2: Simulate a Zomato-style food order process: insert two new items into an OrderItems table, then use ROLLBACK to undo the changes before committing. Check that no new items remain in the table after rollback.

Ans:

```
USE mydb;
DROP TABLE IF EXISTS OrderItems;
CREATE TABLE OrderItems (
   item_id INT PRIMARY KEY,
   order_id INT,
   item_name VARCHAR(100),
   quantity INT,
   price DECIMAL(10,2)
) ENGINE=InnoDB;
START TRANSACTION;
INSERT INTO OrderItems
(item_id, order_id, item_name, quantity, price)
VALUES
(901, 500, 'Pizza', 1, 250.00),
(902, 500, 'Burger', 2, 180.00);
SELECT * FROM OrderItems;
ROLLBACK;
SELECT * FROM OrderItems;
```

item_id	order_id	item_name	quantity	price
901	500	Pizza	1	250.00
902	500	Burger	2	180.00
NULL	NULL	NULL	NULL	NULL

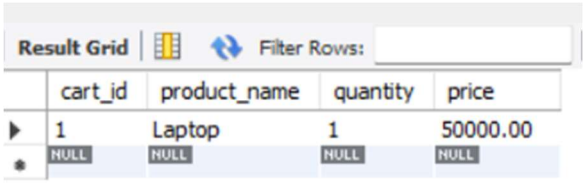
Question3: In a Flipkart-like shopping cart scenario, use SAVEPOINT to mark a point after adding a product to the Cart table, add another product, then use ROLLBACK TO SAVEPOINT to undo only the last addition. Show the final contents of the Cart table.

Ans:

```
USE mydb;
DROP TABLE IF EXISTS Cart;
```

```
CREATE TABLE Cart (
   cart_id INT PRIMARY KEY,
   product_name VARCHAR(100),
   quantity INT,
   price DECIMAL(10,2)
) ENGINE=InnoDB;

START TRANSACTION;
INSERT INTO Cart (cart_id, product_name, quantity, price)
VALUES (1, 'Laptop', 1, 50000.00);
SAVEPOINT product_added;
INSERT INTO Cart (cart_id, product_name, quantity, price)
VALUES (2, 'Mobile', 1, 20000.00);
ROLLBACK TO SAVEPOINT product_added;
SELECT * FROM Cart;
```



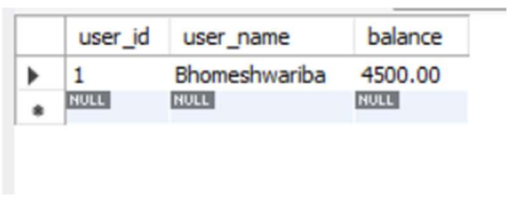
cart_id	product_name	quantity	price
1	Laptop	1	50000.00
NULL	NULL	NULL	NULL

Question4: Write a trigger on a Wallet table that automatically deducts the purchase amount from the user's balance whenever a new transaction is inserted, similar to how Paytm updates wallet balance after a payment.

Ans:

```
USE mydb;
DROP TABLE IF EXISTS Transactions;
DROP TABLE IF EXISTS Wallet;
CREATE TABLE Wallet (
   user_id INT PRIMARY KEY,
```

```
  user_name VARCHAR(100),
  balance DECIMAL(10,2)
);
CREATE TABLE Transactions (
  transaction_id INT PRIMARY KEY,
  user_id INT,
  purchase_amount DECIMAL(10,2)
);
INSERT INTO Wallet VALUES
(1, 'Bhomeshwariba', 5000.00);
DELIMITER //
CREATE TRIGGER deduct_wallet_balance
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
  UPDATE Wallet
  SET balance = balance - NEW.purchase_amount
  WHERE user_id = NEW.user_id;
END //
DELIMITER ;
INSERT INTO Transactions
VALUES (101, 1, 500.00);
SELECT * FROM Wallet;
```



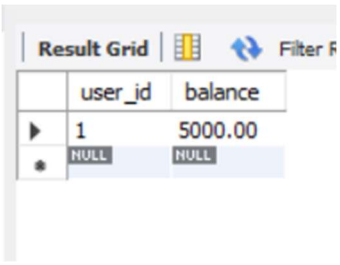
user_id	user_name	balance
1	Bhomeshwariba	4500.00
NULL	NULL	NULL

Question5: Use ChatGPT to generate a SQL test scenario where a trigger incorrectly updates a user's balance after an expense entry. Paste the scenario and your corrected trigger code, explaining how you fixed the error.

Hint: Ask ChatGPT for a buggy trigger example and a test case that exposes the bug.

Ans:

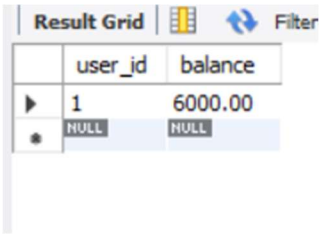
```
USE mydb;
CREATE TABLE Wallet (
  user_id INT PRIMARY KEY,
  balance DECIMAL(10,2)
);
CREATE TABLE Expense (
  expense_id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT,
  amount DECIMAL(10,2)
);
INSERT INTO Wallet (user_id, balance)
VALUES (1, 5000);
SELECT * FROM Wallet;
```



- Buggy Trigger:

```
DELIMITER //
CREATE TRIGGER update_balance
AFTER INSERT ON Expense
FOR EACH ROW
```

```
BEGIN
  UPDATE Wallet
  SET balance = balance + NEW.amount
  WHERE user_id = NEW.user_id;
END//
DELIMITER ;
INSERT INTO Expense (user_id, amount)
VALUES (1, 1000);
SELECT * FROM Wallet;
```




```
DROP TRIGGER IF EXISTS update_balance;

DELIMITER //

CREATE TRIGGER update_balance

AFTER INSERT ON Expense

FOR EACH ROW

BEGIN

  UPDATE Wallet

  SET balance = balance - NEW.amount

  WHERE user_id = NEW.user_id;

END//

DELIMITER ;

UPDATE Wallet

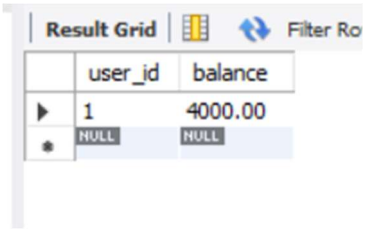
SET balance = 5000

WHERE user_id = 1;

INSERT INTO Expense (user_id, amount)

VALUES (1, 1000);

SELECT * FROM Wallet;
```


Result Grid
Filter Ro
user_id
balance
1
4000.00
NULL
NULL